



# CHRONOS - Technical Specification (SPEC.md)

**Mục đích:** File này là spec kỹ thuật đầy đủ để AI (Claude Code / Cursor / Copilot) có thể sinh ra toàn bộ codebase mà không cần đoán. Mọi class, hàm, format, edge case đều được định nghĩa rõ ràng.

---

## 0. Thông Tin Dự Án

- **Tên:** Chronos - Personal Task & Deadline Manager
  - **Ngôn ngữ:** C++17
  - **Compiler:** g++ 9.0+ / MSVC 2019+
  - **Build system:** CMake 3.15+
  - **Platform:** Cross-platform (Windows / Linux / macOS)
  - **Dependencies:** Chỉ STL chuẩn, không dùng thư viện ngoài.
  - **Test framework:** GoogleTest (gtest) - fetch qua CMake FetchContent.
- 

## 1. Cấu Trúc Thư Mục

```
chronos/  
├── CMakeLists.txt  
├── README.md  
├── SPEC.md  
├── include/  
│   ├── Date.h  
│   ├── Task.h  
│   ├── Project.h  
│   ├── Exceptions.h  
│   ├── UrgencyCalculator.h  
│   ├── TaskFilter.h  
│   ├── UndoManager.h  
│   ├── FileHandler.h  
│   ├── ConsoleColor.h  
│   └── Menu.h
```

```
├─ src/
│   ├── Date.cpp
│   ├── Task.cpp
│   ├── Project.cpp
│   ├── UrgencyCalculator.cpp
│   ├── TaskFilter.cpp
│   ├── UndoManager.cpp
│   ├── FileHandler.cpp
│   ├── ConsoleColor.cpp
│   ├── Menu.cpp
│   └─ main.cpp
├─ tests/
│   ├── CMakeLists.txt
│   ├── unit/
│   │   ├── test_date.cpp
│   │   ├── test_task.cpp
│   │   ├── test_project.cpp
│   │   ├── test_urgency.cpp
│   │   ├── test_filter.cpp
│   │   ├── test_undo.cpp
│   │   ├── test_filehandler.cpp
│   │   └─ test_exceptions.cpp
│   └─ integration/
│       ├── test_full_workflow.cpp
│       ├── test_undo_workflow.cpp
│       ├── test_persistence.cpp
│       └─ test_error_scenarios.cpp
└─ data/
    └─ chronos_save.csv (auto-generated)
```

---

## 2. Convention & Quy Ước Chung

- **Naming:**
  - **Class:** PascalCase (vd: Task , UndoManager )
  - **Method/variable:** camelCase (vd: addTask , dueDate )
  - **Constant/Enum value:** UPPER\_SNAKE\_CASE (vd: MAX\_TASKS , HIGH )
  - **File:** trùng tên class (vd: Task.h , Task.cpp )
- **Memory:** Ưu tiên giá trị (value semantics). Không dùng raw pointer; nếu cần dùng smart pointer ( std::unique\_ptr / std::shared\_ptr ).

- **Error handling:** Dùng **exception** (custom class kế thừa `std::exception`), KHÔNG dùng error code.
  - **Const-correctness:** Mọi getter và hàm không mutate đều phải `const`.
  - **Header guard:** Dùng `#pragma once`.
  - **Encoding:** UTF-8.
- 

## 3. Module 1: Core Domain

### 3.1. Class `Date`

**File:** `include/Date.h`, `src/Date.cpp`

```
class Date {
private:
    int day;    // 1-31
    int month;  // 1-12
    int year;   // 1900-2100

public:
    // Constructors
    Date(); // Mặc định: 01/01/2000
    Date(int day, int month, int year); // Throw InvalidDateException nếu sa

    // Getters
    int getDay() const;
    int getMonth() const;
    int getYear() const;

    // Setters (có validate)
    void setDay(int d);
    void setMonth(int m);
    void setYear(int y);

    // Methods
    bool isValid() const; // Kiểm tra ngày hợp lệ (kể cả năm n
    int daysBetween(const Date& other) const; // Số ngày giữa 2 date (có thể âm)
    Date addDays(int n) const; // Trả về Date mới = this + n ngày
    std::string toString() const; // Format "DD/MM/YYYY"
    static Date today(); // Lấy ngày hôm nay từ system clock
    static Date fromString(const std::string& s); // Parse "DD/MM/YYYY"

    // Operator overload
```

```

    bool operator<(const Date& other) const;
    bool operator>(const Date& other) const;
    bool operator==(const Date& other) const;
    bool operator!=(const Date& other) const;
    bool operator<=(const Date& other) const;
    bool operator>=(const Date& other) const;
    friend std::ostream& operator<<(std::ostream& os, const Date& d);
};

```

**Quy tắc năm nhuận:** chia hết cho 4 VÀ (không chia hết cho 100 HOẶC chia hết cho 400).

---

### 3.2. Enum `Priority` & `Status`

**File:** `include/Task.h`

```

enum class Priority {
    LOW = 1,
    MEDIUM = 2,
    HIGH = 3,
    CRITICAL = 4
};

enum class Status {
    PENDING,
    IN_PROGRESS,
    DONE
};

// Helper functions
std::string priorityToString(Priority p);
Priority priorityFromString(const std::string& s);
std::string statusToString(Status s);
Status statusFromString(const std::string& s);

```

---

### 3.3. Class `Task`

**File:** `include/Task.h`, `src/Task.cpp`

```

class Task {
private:
    int id;
    std::string title;

```

```

    std::string description;
    Date startDate;
    Date dueDate;
    Priority priority;
    Status status;

    static int nextId; // Auto-increment ID

public:
    // Constructors
    Task();
    Task(const std::string& title,
          const std::string& description,
          const Date& startDate,
          const Date& dueDate,
          Priority priority);

    // Throw TimeParadoxException nếu dueDate < startDate

    Task(const Task& other);           // Copy constructor
    Task& operator=(const Task& other); // Copy assignment
    ~Task();

    // Getters
    int getId() const;
    std::string getTitle() const;
    std::string getDescription() const;
    Date getStartDate() const;
    Date getDueDate() const;
    Priority getPriority() const;
    Status getStatus() const;

    // Setters (có validate)
    void setTitle(const std::string& t);
    void setDescription(const std::string& d);
    void setStartDate(const Date& d);    // Validate: startDate <= dueDate
    void setDueDate(const Date& d);      // Validate: dueDate >= startDate
    void setPriority(Priority p);
    void setStatus(Status s);

    // Business methods
    int daysLeft() const;                // Số ngày còn lại đến deadline (có thể
    bool isOverdue() const;              // dueDate < today && status != DONE
    void display() const;                // In ra console

    // Operator overload
    bool operator<(const Task& other) const; // So sánh urgency, ưu tiên cao ->

```

```

    bool operator==(const Task& other) const;    // So sánh theo id
    Task& operator++();                          // Prefix: gia hạn dueDate +1 ngày
    Task operator++(int);                       // Postfix: gia hạn dueDate +1 ngày
    friend std::ostream& operator<<(std::ostream& os, const Task& t);

    // Static
    static void resetIdCounter();                // Dùng cho test
};

```

### Quy tắc operator< :

1. So sánh Priority (CRITICAL > HIGH > MEDIUM > LOW).
2. Nếu bằng, so sánh dueDate (gần hơn → "nhỏ hơn" → đứng trước).
3. Trả về true nếu this "khẩn cấp hơn" other.

**LƯU Ý:** operator< ngược với so sánh số học thông thường — task khẩn cấp hơn được coi là "nhỏ hơn" để std::sort đưa lên đầu.

---

## 3.4. Class Project

**File:** include/Project.h, src/Project.cpp

```

class Project {
private:
    int projectId;
    std::string projectName;
    std::string description;
    std::vector<Task> tasks;

    static const int MAX_TASKS = 9999;
    static int nextProjectId;

public:
    // Constructors
    Project();
    Project(const std::string& name, const std::string& description);

    // Getters
    int getProjectId() const;
    std::string getProjectName() const;
    std::string getDescription() const;
    const std::vector<Task>& getAllTasks() const;
    int getTaskCount() const;

```

```

// Setters
void setProjectName(const std::string& name);
void setDescription(const std::string& desc);

// Task management
void addTask(const Task& t);           // Throw BufferOverflowException nếu >=
bool removeTask(int taskId);          // Trả về true nếu xóa thành công
Task* getTaskById(int taskId);        // Trả về nullptr nếu không tìm thấy
bool updateTask(int taskId, const Task& newTask);

// Display
void displayProject() const;
void displayAllTasks() const;         // In bảng đẹp với iomanip

// Comparison (cho UndoManager so sánh state)
bool operator==(const Project& other) const;
};

```

---

## 4. Module 2: Logic & Tính Toán

### 4.1. Class `UrgencyCalculator`

**File:** `include/UrgencyCalculator.h`, `src/UrgencyCalculator.cpp`

```

class UrgencyCalculator {
public:
    // Công thức: urgency = (priority * 10) + (100 / (daysLeft + 1))
    // Nếu daysLeft < 0 (overdue) -> urgency += 1000 (ưu tiên cực cao)
    // Nếu status == DONE -> urgency = 0
    static double calculate(const Task& task);

    // Sắp xếp danh sách task theo urgency giảm dần
    static void sortByUrgency(std::vector<Task>& tasks);

    // Sắp xếp theo deadline tăng dần (gần nhất trước)
    static void sortByDeadline(std::vector<Task>& tasks);

    // Sắp xếp theo priority giảm dần
    static void sortByPriority(std::vector<Task>& tasks);
};

```

**Công thức chi tiết:**

```
if (status == DONE):
    return 0.0
daysLeft = task.daysLeft()
base = (int)priority * 10
if (daysLeft < 0):
    return base + 1000.0 // overdue boost
else:
    return base + (100.0 / (daysLeft + 1))
```

---

## 4.2. Class `TaskFilter`

**File:** `include/TaskFilter.h`, `src/TaskFilter.cpp`

```
class TaskFilter {
public:
    // Tìm theo từ khóa trong title hoặc description (case-insensitive)
    static std::vector<Task> searchByKeyword(const std::vector<Task>& tasks,
                                             const std::string& keyword);

    // Lọc theo priority
    static std::vector<Task> filterByPriority(const std::vector<Task>& tasks,
                                             Priority p);

    // Lọc theo status
    static std::vector<Task> filterByStatus(const std::vector<Task>& tasks,
                                             Status s);

    // Lọc task có dueDate trong khoảng [from, to]
    static std::vector<Task> filterByDateRange(const std::vector<Task>& tasks,
                                                const Date& from,
                                                const Date& to);

    // Lấy tất cả task quá hạn
    static std::vector<Task> getOverdueTasks(const std::vector<Task>& tasks);
};
```

---

## 4.3. Custom Exceptions

**File:** `include/Exceptions.h`

```
#include <stdexcept>
```



```

class TimeParadoxException : public std::runtime_error {
public:
    TimeParadoxException()
        : std::runtime_error("Lỗi: Nghịch lý thời gian! Ngày hết hạn không thể tr
    };

class BufferOverflowException : public std::runtime_error {
public:
    BufferOverflowException()
        : std::runtime_error("Lỗi: Vượt quá giới hạn lưu trữ! Số nhiệm vụ tối đa
    };

class InvalidDateException : public std::runtime_error {
public:
    InvalidDateException(const std::string& msg)
        : std::runtime_error("Lỗi ngày không hợp lệ: " + msg) {}
    };

class TaskNotFoundException : public std::runtime_error {
public:
    TaskNotFoundException(int id)
        : std::runtime_error("Không tìm thấy nhiệm vụ với ID = " + std::to_string
    };

class FileIOException : public std::runtime_error {
public:
    FileIOException(const std::string& msg)
        : std::runtime_error("Lỗi I/O file: " + msg) {}
    };

class UndoStackEmptyException : public std::runtime_error {
public:
    UndoStackEmptyException()
        : std::runtime_error("Không còn thao tác nào để hoàn tác.") {}
    };

```

---

## 5. Module 3: Persistence & Undo

### 5.1. Class `UndoManager`

**File:** `include/UndoManager.h`, `src/UndoManager.cpp`

```

class UndoManager {
private:

```

```

    std::stack<Project> history;
    static const int MAX_UNDO_DEPTH = 50;

public:
    UndoManager();

    // Lưu snapshot trước khi thay đổi
    void saveState(const Project& currentState);

    // Khôi phục state gần nhất. Throw UndoStackEmptyException nếu stack rỗng.
    Project undo();

    // Xóa toàn bộ history
    void clear();

    // Số state đang lưu
    size_t size() const;
    bool isEmpty() const;
};

```

### Quy tắc:

- Trước MỖI thao tác mutate ( addTask , removeTask , updateTask , gia hạn, đổi status...) → gọi saveState() .
- Nếu size() > MAX\_UNDO\_DEPTH , xóa state cũ nhất (cần dùng std::deque thay cho std::stack hoặc tự implement).
- **GHI CHÚ THIẾT KẾ:** Mặc dù yêu cầu là std::stack , để giới hạn 50 phần tử cần dùng std::deque làm container ngầm: std::stack<Project, std::deque<Project>> . Nhưng std::stack không cho truy cập đáy. Giải pháp: dùng std::deque<Project> trực tiếp và mô phỏng API stack ( push\_back / pop\_back ). Comment rõ trong code lý do.

## 5.2. Class FileHandler

**File:** include/FileHandler.h , src/FileHandler.cpp

```

class FileHandler {
public:
    // Lưu project ra file CSV
    static void saveProject(const Project& project, const std::string& filepath);

    // Đọc project từ file CSV
    static Project loadProject(const std::string& filepath);

```

```

        // Kiểm tra file tồn tại
        static bool fileExists(const std::string& filepath);
};

```

### Format file CSV ( data/chronos\_save.csv ):

```

# PROJECT
projectId, projectName, description
1, My Project, Description here

# TASKS
taskId, title, description, startDate, dueDate, priority, status
1, Buy groceries, Milk and eggs, 01/05/2026, 03/05/2026, HIGH, PENDING
2, Write report, Q2 financial report, 02/05/2026, 10/05/2026, CRITICAL, IN_PROGRESS

```

### Quy tắc:

- Dấu `,` trong text được escape bằng cách bao trong dấu `"` .
- Khi đọc file lỗi → throw `FileIOException` .
- Khi file không tồn tại → throw `FileIOException` .

## 6. Module 4: CLI & Menu

### 6.1. Class `Menu`

**File:** include/Menu.h , src/Menu.cpp

```

class Menu {
private:
    Project currentProject;
    UndoManager undoManager;
    std::string saveFilePath = "data/chronos_save.csv";
    bool isDirty = false;           // True nếu có thay đổi chưa lưu

public:
    Menu();

    // Vòng lặp chính
    void run();

private:

```

```

// Hiển thị
void displayMainMenu();
void displayTaskTable(const std::vector<Task>& tasks);

// Parse command (số HOẶC từ khóa)
int parseCommand(const std::string& input);

// Xử lý lựa chọn
void handleCreateProject();
void handleAddTask();
void handleViewTasks();
void handleSortByUrgency();
void handleExtendDeadline();      // Dùng operator++
void handleUndo();
void handleSaveLoad();
void handleSearch();
void handleDeleteTask();
void handleUpdateStatus();
void handleStressTest();          // Demo Buffer Overflow
void handleExit();                // Auto-save + cảnh báo

// Helpers
int readInt(const std::string& prompt);
std::string readString(const std::string& prompt);
Date readDate(const std::string& prompt);
Priority readPriority();
bool readYesNo(const std::string& prompt);
void pressEnterToContinue();

// Đánh dấu state đã thay đổi
void markDirty() { isDirty = true; }
void markClean() { isDirty = false; }
};

```

### Quy tắc nhập command (số HOẶC từ khóa):

- User có thể gõ số ( 1 , 2 , ...) hoặc từ khóa tương ứng.
- Bảng mapping từ khóa → số (case-insensitive):

| Số | Từ khóa hỗ trợ   |
|----|------------------|
| 1  | new , create     |
| 2  | add              |
| 3  | list , view , ls |

|    |                      |
|----|----------------------|
| 4  | sort                 |
| 5  | extend               |
| 6  | undo                 |
| 7  | save , load          |
| 8  | search , find        |
| 9  | delete , remove , rm |
| 10 | status , update      |
| 11 | stress               |
| 0  | exit , quit , q      |

- Hàm `parseCommand()` chuyển input về số tương ứng. Nếu không nhận diện được → trả về -1 (in lỗi và prompt lại).

## 6.2. Format Menu (output mẫu)

```

|| CHRONOS - Task Manager v1.0 ||

```

1. Tạo dự án mới
2. Thêm nhiệm vụ
3. Xem danh sách nhiệm vụ
4. Sắp xếp theo độ khẩn cấp
5. Gia hạn deadline (+1 ngày)
6. Hoàn tác (Undo)
7. Lưu / Tải dữ liệu
8. Tìm kiếm nhiệm vụ
9. Xóa nhiệm vụ
10. Cập nhật trạng thái
11. [DEMO] Tràn bộ đệm 10000 task
0. Thoát

---

Lựa chọn của bạn:

## 6.3. Format bảng task (dùng `iomanip` + tô màu ANSI)

| ID | Tiêu đề       | Bắt đầu    | Hết hạn    | Ưu tiên  | Trạng thái  |
|----|---------------|------------|------------|----------|-------------|
| 1  | Buy groceries | 01/05/2026 | 03/05/2026 | HIGH     | PENDING     |
| 2  | Write report  | 02/05/2026 | 10/05/2026 | CRITICAL | IN_PROGRESS |

6.4. Tô màu console (ANSI escape codes)

File: include/ConsoleColor.h

```
namespace ConsoleColor {
    constexpr const char* RESET      = "\033[0m";
    constexpr const char* RED        = "\033[31m";
    constexpr const char* GREEN      = "\033[32m";
    constexpr const char* YELLOW     = "\033[33m";
    constexpr const char* BLUE       = "\033[34m";
    constexpr const char* MAGENTA    = "\033[35m";
    constexpr const char* CYAN       = "\033[36m";
    constexpr const char* BOLD       = "\033[1m";

    // Bật ANSI trên Windows 10+ (gọi 1 lần khi khởi động)
    void enableAnsiOnWindows();
}
```

Quy tắc tô màu task khi hiển thị:

| Điều kiện                                      | Màu cả dòng                                                                                      |
|------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>status == DONE</code>                    |  GREEN      |
| <code>isOverdue() == true</code> (quá hạn)     |  RED        |
| <code>priority == CRITICAL</code> && chưa done |  RED + BOLD |
| <code>priority == HIGH</code> && chưa done     |  YELLOW     |
| <code>urgency &gt;= 50</code> (gần deadline)   |  YELLOW     |
| Còn lại                                        | (không màu)                                                                                      |

Lưu ý cross-platform:

- Linux/macOS: ANSI hoạt động mặc định.

- Windows 10+: Cần gọi `enableAnsiOnWindows()` trong `main()` để bật virtual terminal.
- Windows cũ: Code vẫn chạy nhưng màu không hiển thị (in escape sequence raw) — chấp nhận được.

## 6.5. Auto-save & quản lý phiên làm việc

### Quy tắc:

- Khi khởi động: nếu file `data/chronos_save.csv` tồn tại → tự động `loadProject()` vào `currentProject`. Lỗi đọc file → in cảnh báo, khởi tạo project rỗng.
- Mọi thao tác mutate ( `addTask`, `removeTask`, `updateTask`, gia hạn, đổi status, đổi tên project) → `markDirty()`.
- Mọi thao tác `saveProject()` thành công → `markClean()`.
- Khi chọn lệnh Thoát (option 0):

1. Nếu `isDirty == true`:

- Hỏi: *"Bạn có thay đổi chưa lưu. Lưu trước khi thoát? (Y/N/Cancel)"*
- Y → gọi `saveProject()` rồi exit.
- N → exit không lưu.
- Cancel → quay lại menu chính.

2. Nếu `isDirty == false`:

- Auto-save (ghi đè file để cập nhật timestamp/format) rồi exit.

- Khi nhận `SIGINT` (Ctrl+C): cố gắng auto-save bằng signal handler trước khi exit (best-effort, có thể bỏ qua nếu phức tạp).

### Pseudo-code `handleExit()`:

```
void Menu::handleExit() {
    if (isDirty) {
        std::cout << "Bạn có thay đổi chưa lưu. Lưu trước khi thoát? (Y/N/C): ";
        std::string ans = readString("");
        std::transform(ans.begin(), ans.end(), ans.begin(), ::tolower);
        if (ans == "y" || ans == "yes") {
            FileHandler::saveProject(currentProject, saveFilePath);
            std::cout << "Đã lưu. Tạm biệt!\n";
            std::exit(0);
        } else if (ans == "n" || ans == "no") {
```

```

        std::cout << "Thoát không lưu. Tạm biệt!\n";
        std::exit(0);
    } else {
        // Cancel: quay lại menu
        return;
    }
} else {
    // Không có thay đổi: auto-save im lặng
    try {
        FileHandler::saveProject(currentProject, saveFilePath);
    } catch (...) { /* bỏ qua */ }
    std::cout << "Tạm biệt!\n";
    std::exit(0);
}
}

```

---

## 7. Unit Tests

**Framework:** GoogleTest. Mỗi test file độc lập, có `main()` từ `gtest_main`.

### 7.1. tests/unit/test\_date.cpp

```

TEST(DateTest, DefaultConstructor) {
    Date d;
    EXPECT_EQ(d.getDay(), 1);
    EXPECT_EQ(d.getMonth(), 1);
    EXPECT_EQ(d.getYear(), 2000);
}

TEST(DateTest, ValidDate) {
    Date d(15, 6, 2026);
    EXPECT_TRUE(d.isValid());
}

TEST(DateTest, InvalidDayThrows) {
    EXPECT_THROW(Date(32, 1, 2026), InvalidDateException);
    EXPECT_THROW(Date(0, 1, 2026), InvalidDateException);
}

TEST(DateTest, InvalidMonthThrows) {
    EXPECT_THROW(Date(1, 13, 2026), InvalidDateException);
    EXPECT_THROW(Date(1, 0, 2026), InvalidDateException);
}

```



```

TEST(DateTest, LeapYearFebruary) {
    EXPECT_NO_THROW(Date(29, 2, 2024)); // Năm nhuận
    EXPECT_THROW(Date(29, 2, 2023), InvalidDateException); // Không nhuận
    EXPECT_NO_THROW(Date(29, 2, 2000)); // Chia hết cho 400
    EXPECT_THROW(Date(29, 2, 1900), InvalidDateException); // Chia hết 100 nhưng
}

TEST(DateTest, DaysBetween) {
    Date d1(1, 1, 2026);
    Date d2(11, 1, 2026);
    EXPECT_EQ(d1.daysBetween(d2), 10);
    EXPECT_EQ(d2.daysBetween(d1), -10);
}

TEST(DateTest, AddDays) {
    Date d(28, 2, 2024);
    Date result = d.addDays(2);
    EXPECT_EQ(result.getDay(), 1);
    EXPECT_EQ(result.getMonth(), 3);
    EXPECT_EQ(result.getYear(), 2024);
}

TEST(DateTest, ToStringFormat) {
    Date d(5, 7, 2026);
    EXPECT_EQ(d.toString(), "05/07/2026");
}

TEST(DateTest, FromString) {
    Date d = Date::fromString("15/06/2026");
    EXPECT_EQ(d.getDay(), 15);
    EXPECT_EQ(d.getMonth(), 6);
    EXPECT_EQ(d.getYear(), 2026);
}

TEST(DateTest, ComparisonOperators) {
    Date d1(1, 1, 2026);
    Date d2(2, 1, 2026);
    EXPECT_TRUE(d1 < d2);
    EXPECT_TRUE(d2 > d1);
    EXPECT_TRUE(d1 != d2);
    EXPECT_FALSE(d1 == d2);
}

```

## 7.2. tests/unit/test\_task.cpp

```

TEST(TaskTest, ValidConstruction) {

```

```

        Task t("Test", "Desc", Date(1,5,2026), Date(5,5,2026), Priority::HIGH);
        EXPECT_EQ(t.getTitle(), "Test");
        EXPECT_EQ(t.getPriority(), Priority::HIGH);
        EXPECT_EQ(t.getStatus(), Status::PENDING);
    }

TEST(TaskTest, TimeParadoxOnConstruction) {
    EXPECT_THROW(
        Task("Bad", "Desc", Date(5,5,2026), Date(1,5,2026), Priority::LOW),
        TimeParadoxException
    );
}

TEST(TaskTest, TimeParadoxOnSetDueDate) {
    Task t("Test", "Desc", Date(1,5,2026), Date(5,5,2026), Priority::HIGH);
    EXPECT_THROW(t.setDueDate(Date(31,12,2025)), TimeParadoxException);
}

TEST(TaskTest, AutoIncrementId) {
    Task::resetIdCounter();
    Task t1("A", "", Date(1,1,2026), Date(2,1,2026), Priority::LOW);
    Task t2("B", "", Date(1,1,2026), Date(2,1,2026), Priority::LOW);
    EXPECT_EQ(t2.getId(), t1.getId() + 1);
}

TEST(TaskTest, OperatorPrefixIncrement) {
    Task t("Test", "", Date(1,5,2026), Date(5,5,2026), Priority::HIGH);
    ++t;
    EXPECT_EQ(t.getDueDate(), Date(6,5,2026));
}

TEST(TaskTest, OperatorPostfixIncrement) {
    Task t("Test", "", Date(1,5,2026), Date(5,5,2026), Priority::HIGH);
    Task old = t++;
    EXPECT_EQ(old.getDueDate(), Date(5,5,2026));
    EXPECT_EQ(t.getDueDate(), Date(6,5,2026));
}

TEST(TaskTest, LessOperatorByPriority) {
    Task t1("A", "", Date(1,5,2026), Date(10,5,2026), Priority::CRITICAL);
    Task t2("B", "", Date(1,5,2026), Date(10,5,2026), Priority::LOW);
    EXPECT_TRUE(t1 < t2); // CRITICAL "nhỏ hơn" để sort lên đầu
}

TEST(TaskTest, LessOperatorByDeadlineWhenSamePriority) {
    Task t1("A", "", Date(1,5,2026), Date(5,5,2026), Priority::HIGH);
    Task t2("B", "", Date(1,5,2026), Date(10,5,2026), Priority::HIGH);

```

```

    EXPECT_TRUE(t1 < t2);
}

TEST(TaskTest, IsOverdue) {
    Task t("Old", "", Date(1,1,2020), Date(2,1,2020), Priority::LOW);
    EXPECT_TRUE(t.isOverdue());
    t.setStatus(Status::DONE);
    EXPECT_FALSE(t.isOverdue());
}

```

### 7.3. tests/unit/test\_project.cpp

```

TEST(ProjectTest, AddTaskIncreasesCount) {
    Project p("Test", "Desc");
    Task t("A", "", Date(1,5,2026), Date(5,5,2026), Priority::HIGH);
    p.addTask(t);
    EXPECT_EQ(p.getTaskCount(), 1);
}

TEST(ProjectTest, RemoveExistingTask) {
    Project p("Test", "Desc");
    Task t("A", "", Date(1,5,2026), Date(5,5,2026), Priority::HIGH);
    p.addTask(t);
    int id = t.getId();
    EXPECT_TRUE(p.removeTask(id));
    EXPECT_EQ(p.getTaskCount(), 0);
}

TEST(ProjectTest, RemoveNonExistentTaskReturnsFalse) {
    Project p("Test", "Desc");
    EXPECT_FALSE(p.removeTask(99999));
}

TEST(ProjectTest, BufferOverflowAt9999) {
    Project p("Test", "Desc");
    for (int i = 0; i < 9999; i++) {
        Task t("T" + std::to_string(i), "", Date(1,5,2026), Date(5,5,2026), Priority::LOW);
        p.addTask(t);
    }
    EXPECT_EQ(p.getTaskCount(), 9999);
    Task overflow("Overflow", "", Date(1,5,2026), Date(5,5,2026), Priority::LOW);
    EXPECT_THROW(p.addTask(overflow), BufferOverflowException);
}

TEST(ProjectTest, GetTaskById) {
    Project p("Test", "Desc");
}

```

```

    Task t("Find me", "", Date(1,5,2026), Date(5,5,2026), Priority::HIGH);
    p.addTask(t);
    Task* found = p.getTaskById(t.getId());
    ASSERT_NE(found, nullptr);
    EXPECT_EQ(found->getTitle(), "Find me");
}

```

#### 7.4. tests/unit/test\_urgency.cpp

```

TEST(UrgencyTest, DoneTaskHasZeroUrgency) {
    Task t("Done", "", Date::today(), Date::today().addDays(5), Priority::CRITICAL);
    t.setStatus(Status::DONE);
    EXPECT_DOUBLE_EQ(UrgencyCalculator::calculate(t), 0.0);
}

TEST(UrgencyTest, OverdueTaskGetsBoost) {
    Date past = Date::today().addDays(-5);
    Task t("Old", "", past.addDays(-1), past, Priority::LOW);
    double u = UrgencyCalculator::calculate(t);
    EXPECT_GT(u, 1000.0);
}

TEST(UrgencyTest, HigherPriorityHigherUrgency) {
    Date today = Date::today();
    Task low("L", "", today, today.addDays(5), Priority::LOW);
    Task crit("C", "", today, today.addDays(5), Priority::CRITICAL);
    EXPECT_GT(UrgencyCalculator::calculate(crit), UrgencyCalculator::calculate(low));
}

TEST(UrgencyTest, SortByUrgencyDescending) {
    std::vector<Task> tasks = {
        Task("Low", "", Date::today(), Date::today().addDays(10), Priority::LOW),
        Task("Crit", "", Date::today(), Date::today().addDays(1), Priority::CRITICAL),
        Task("Med", "", Date::today(), Date::today().addDays(5), Priority::MEDIUM);
    };
    UrgencyCalculator::sortByUrgency(tasks);
    EXPECT_EQ(tasks[0].getTitle(), "Crit");
}

```

#### 7.5. tests/unit/test\_filter.cpp

```

TEST(FilterTest, SearchByKeywordCaseInsensitive) {
    std::vector<Task> tasks = {
        Task("Buy Milk", "Grocery", Date::today(), Date::today().addDays(1), Priority::LOW);
    };
}

```

```

        Task("Read book", "Fun", Date::today(), Date::today().addDays(1), Priorit
    };
    auto result = TaskFilter::searchByKeyword(tasks, "MILK");
    EXPECT_EQ(result.size(), 1);
    EXPECT_EQ(result[0].getTitle(), "Buy Milk");
}

TEST(FilterTest, FilterByPriority) {
    std::vector<Task> tasks = {
        Task("A", "", Date::today(), Date::today().addDays(1), Priority::HIGH),
        Task("B", "", Date::today(), Date::today().addDays(1), Priority::LOW)
    };
    auto result = TaskFilter::filterByPriority(tasks, Priority::HIGH);
    EXPECT_EQ(result.size(), 1);
}

TEST(FilterTest, GetOverdueTasks) {
    std::vector<Task> tasks = {
        Task("Old", "", Date(1,1,2020), Date(2,1,2020), Priority::LOW),
        Task("Future", "", Date::today(), Date::today().addDays(10), Priority::LO
    };
    auto result = TaskFilter::getOverdueTasks(tasks);
    EXPECT_EQ(result.size(), 1);
    EXPECT_EQ(result[0].getTitle(), "Old");
}

```

## 7.6. tests/unit/test\_undo.cpp

```

TEST(UndoTest, EmptyStackThrows) {
    UndoManager um;
    EXPECT_THROW(um.undo(), UndoStackEmptyException);
}

TEST(UndoTest, SaveAndRestore) {
    UndoManager um;
    Project p1("V1", "");
    um.saveState(p1);
    Project p2("V2", "");
    Project restored = um.undo();
    EXPECT_EQ(restored.getProjectName(), "V1");
}

TEST(UndoTest, MaxDepthLimit) {
    UndoManager um;
    for (int i = 0; i < 60; i++) {
        Project p("V" + std::to_string(i), "");
    }
}

```

```

        um.saveState(p);
    }
    EXPECT_LE(um.size(), 50);
}

TEST(UndoTest, ClearEmptiesStack) {
    UndoManager um;
    um.saveState(Project("A", ""));
    um.saveState(Project("B", ""));
    um.clear();
    EXPECT_TRUE(um.isEmpty());
}

```

## 7.7. tests/unit/test\_filehandler.cpp

```

TEST(FileHandlerTest, SaveAndLoadRoundtrip) {
    Project p("Original", "Desc");
    Task t("Task1", "Body", Date(1,5,2026), Date(5,5,2026), Priority::HIGH);
    p.addTask(t);

    FileHandler::saveProject(p, "test_save.csv");
    Project loaded = FileHandler::loadProject("test_save.csv");

    EXPECT_EQ(loaded.getProjectName(), "Original");
    EXPECT_EQ(loaded.getTaskCount(), 1);
    EXPECT_EQ(loaded.getAllTasks()[0].getTitle(), "Task1");

    std::remove("test_save.csv");
}

TEST(FileHandlerTest, LoadNonExistentThrows) {
    EXPECT_THROW(FileHandler::loadProject("nonexistent.csv"), FileIOException);
}

TEST(FileHandlerTest, EscapeCommasInText) {
    Project p("Name, with comma", "Desc, with comma");
    Task t("Title, with comma", "Body", Date(1,5,2026), Date(5,5,2026), Priority::HIGH);
    p.addTask(t);

    FileHandler::saveProject(p, "test_comma.csv");
    Project loaded = FileHandler::loadProject("test_comma.csv");

    EXPECT_EQ(loaded.getProjectName(), "Name, with comma");
    EXPECT_EQ(loaded.getAllTasks()[0].getTitle(), "Title, with comma");

    std::remove("test_comma.csv");
}

```

```
}
```

## 7.8. tests/unit/test\_exceptions.cpp

```
TEST(ExceptionTest, TimeParadoxMessage) {
    try {
        throw TimeParadoxException();
    } catch (const std::exception& e) {
        std::string msg(e.what());
        EXPECT_NE(msg.find("Nghịch lý thời gian"), std::string::npos);
    }
}

TEST(ExceptionTest, BufferOverflowMessage) {
    try {
        throw BufferOverflowException();
    } catch (const std::exception& e) {
        std::string msg(e.what());
        EXPECT_NE(msg.find("9999"), std::string::npos);
    }
}
```

---

# 8. Integration Tests

## 8.1. tests/integration/test\_full\_workflow.cpp

```
// Test luồng đầy đủ: tạo project -> thêm task -> sort -> gia hạn -> xem
TEST(IntegrationTest, FullUserWorkflow) {
    Project project("My Day", "Daily tasks");

    // 1. Thêm 3 task với priority khác nhau
    Task t1("Email reply", "", Date::today(), Date::today().addDays(7), Priority:
    Task t2("Submit report", "", Date::today(), Date::today().addDays(2), Priorit
    Task t3("Team meeting", "", Date::today(), Date::today().addDays(1), Priority

    project.addTask(t1);
    project.addTask(t2);
    project.addTask(t3);
    EXPECT_EQ(project.getTaskCount(), 3);

    // 2. Sort theo urgency
    auto tasks = project.getAllTasks();
```

```

    UrgencyCalculator::sortByUrgency(tasks);
    EXPECT_EQ(tasks[0].getTitle(), "Submit report");

    // 3. Gia hạn task đầu tiên
    Task* found = project.getTaskById(t2.getId());
    ASSERT_NE(found, nullptr);
    Date oldDue = found->getDueDate();
    ++(*found);
    EXPECT_EQ(found->getDueDate(), oldDue.addDays(1));

    // 4. Update status
    found->setStatus(Status::DONE);
    EXPECT_DOUBLE_EQ(UrgencyCalculator::calculate(*found), 0.0);
}

```

## 8.2. tests/integration/test\_undo\_workflow.cpp

```

// Kịch bản: thêm task -> undo -> xác nhận task biến mất
TEST(IntegrationTest, UndoAfterAddTask) {
    Project p("Test", "");
    UndoManager um;

    um.saveState(p);
    Task t("New", "", Date::today(), Date::today().addDays(1), Priority::LOW);
    p.addTask(t);
    EXPECT_EQ(p.getTaskCount(), 1);

    p = um.undo();
    EXPECT_EQ(p.getTaskCount(), 0);
}

// Kịch bản: thêm 3 task -> xóa 1 -> undo -> xác nhận task quay lại
TEST(IntegrationTest, UndoAfterRemoveTask) {
    Project p("Test", "");
    UndoManager um;

    Task t1("A", "", Date::today(), Date::today().addDays(1), Priority::LOW);
    Task t2("B", "", Date::today(), Date::today().addDays(1), Priority::LOW);
    p.addTask(t1);
    p.addTask(t2);

    um.saveState(p);
    p.removeTask(t1.getId());
    EXPECT_EQ(p.getTaskCount(), 1);

    p = um.undo();
}

```



```

        EXPECT_EQ(p.getTaskCount(), 2);
    }

    // Multi-step undo
    TEST(IntegrationTest, MultipleUndo) {
        Project p("V0", "");
        UndoManager um;

        um.saveState(p);
        p.setProjectName("V1");
        um.saveState(p);
        p.setProjectName("V2");

        p = um.undo();
        EXPECT_EQ(p.getProjectName(), "V1");

        p = um.undo();
        EXPECT_EQ(p.getProjectName(), "V0");

        EXPECT_THROW(um.undo(), UndoStackEmptyException);
    }

```

### 8.3. tests/integration/test\_persistence.cpp

```

// Save -> load -> sort -> save lại -> load -> check thứ tự giữ nguyên
TEST(IntegrationTest, PersistAfterSort) {
    Project p("Persist", "");
    p.addTask(Task("Low", "", Date::today(), Date::today().addDays(10), Priority
    p.addTask(Task("Crit", "", Date::today(), Date::today().addDays(1), Priority

    FileHandler::saveProject(p, "persist_test.csv");
    Project loaded = FileHandler::loadProject("persist_test.csv");

    EXPECT_EQ(loaded.getTaskCount(), 2);

    std::remove("persist_test.csv");
}

// Round-trip với nhiều task
TEST(IntegrationTest, LargeProjectRoundtrip) {
    Project p("Big", "");
    for (int i = 0; i < 100; i++) {
        p.addTask(Task("T" + std::to_string(i), "Desc",
                        Date::today(), Date::today().addDays(i+1), Priority::MEDIU
    }
}

```

```

    FileHandler::saveProject(p, "big_test.csv");
    Project loaded = FileHandler::loadProject("big_test.csv");

    EXPECT_EQ(loaded.getTaskCount(), 100);
    std::remove("big_test.csv");
}

```

#### 8.4. tests/integration/test\_error\_scenarios.cpp

```

// Demo Time Paradox - bắt buộc cho video
TEST(ErrorScenario, TimeParadoxDemo) {
    Date start(10, 5, 2026);
    Date due(1, 5, 2026); // Trước start

    EXPECT_THROW(
        Task("Paradox", "", start, due, Priority::HIGH),
        TimeParadoxException
    );
}

// Demo Buffer Overflow - bắt buộc cho video
TEST(ErrorScenario, BufferOverflowDemo) {
    Project p("Stress", "");

    int addedCount = 0;
    bool caughtOverflow = false;

    try {
        for (int i = 0; i < 10000; i++) {
            Task t("T" + std::to_string(i), "",
                Date::today(), Date::today().addDays(1), Priority::LOW);
            p.addTask(t);
            addedCount++;
        }
    } catch (const BufferOverflowException& e) {
        caughtOverflow = true;
    }

    EXPECT_TRUE(caughtOverflow);
    EXPECT_EQ(addedCount, 9999);
    EXPECT_EQ(p.getTaskCount(), 9999);
}

// Test rằng sau khi lỗi, system vẫn hoạt động bình thường
TEST(ErrorScenario, RecoveryAfterTimeParadox) {
    Project p("Recover", "");

```

```

    try {
        Task bad("Bad", "", Date(10,5,2026), Date(1,5,2026), Priority::HIGH);
    } catch (const TimeParadoxException&) {
        // expected
    }

    // System vẫn add task tốt
    Task good("Good", "", Date(1,5,2026), Date(10,5,2026), Priority::HIGH);
    EXPECT_NO_THROW(p.addTask(good));
    EXPECT_EQ(p.getTaskCount(), 1);
}

```

## 8.5. tests/integration/test\_menu\_features.cpp

```

// Test parse command: nhập số
TEST(MenuFeature, ParseCommandNumeric) {
    Menu menu;
    EXPECT_EQ(menu.parseCommand("1"), 1);
    EXPECT_EQ(menu.parseCommand("11"), 11);
    EXPECT_EQ(menu.parseCommand("0"), 0);
}

// Test parse command: nhập từ khóa (case-insensitive)
TEST(MenuFeature, ParseCommandKeyword) {
    Menu menu;
    EXPECT_EQ(menu.parseCommand("add"), 2);
    EXPECT_EQ(menu.parseCommand("ADD"), 2);
    EXPECT_EQ(menu.parseCommand("list"), 3);
    EXPECT_EQ(menu.parseCommand("ls"), 3);
    EXPECT_EQ(menu.parseCommand("undo"), 6);
    EXPECT_EQ(menu.parseCommand("exit"), 0);
    EXPECT_EQ(menu.parseCommand("quit"), 0);
    EXPECT_EQ(menu.parseCommand("q"), 0);
}

// Từ khóa không hợp lệ trả về -1
TEST(MenuFeature, ParseCommandInvalid) {
    Menu menu;
    EXPECT_EQ(menu.parseCommand("xyz"), -1);
    EXPECT_EQ(menu.parseCommand(""), -1);
    EXPECT_EQ(menu.parseCommand("99"), -1);
}

// Auto-save: sau khi save, isDirty phải false
TEST(MenuFeature, DirtyFlagAfterSave) {

```

```

Project p("Test", "");
Task t("A", "", Date::today(), Date::today().addDays(1), Priority::LOW);
p.addTask(t);

FileHandler::saveProject(p, "dirty_test.csv");
// Đọc lại từ file → state == file → "clean"
Project loaded = FileHandler::loadProject("dirty_test.csv");
EXPECT_EQ(loaded.getTaskCount(), 1);

std::remove("dirty_test.csv");
}

// Auto-load khi khởi động Menu (file tồn tại)
TEST(MenuFeature, AutoLoadOnStart) {
    // Setup: tạo file save trước
    Project p("Preload", "");
    p.addTask(Task("Existing", "", Date::today(), Date::today().addDays(1), Prior
    FileHandler::saveProject(p, "data/chronos_save.csv");

    // Khi khởi tạo Menu, currentProject phải được load
    Menu menu;
    // Giả sử Menu có getter cho test
    // EXPECT_EQ(menu.getCurrentProject().getTaskCount(), 1);
    // (Hoặc test gián tiếp qua command "list")

    std::remove("data/chronos_save.csv");
}

```

## 8.6. tests/integration/test\_console\_color.cpp

```

// Test rằng ANSI escape không gây crash
TEST(ConsoleColor, ConstantsAreValidAnsi) {
    using namespace ConsoleColor;
    EXPECT_STREQ(RESET, "\033[0m");
    EXPECT_STREQ(RED, "\033[31m");
    EXPECT_STREQ(GREEN, "\033[32m");
}

// Test enableAnsiOnWindows không throw
TEST(ConsoleColor, EnableAnsiNoThrow) {
    EXPECT_NO_THROW(ConsoleColor::enableAnsiOnWindows());
}

```

---

## 9. CMakeLists.txt

### 9.1. Root CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
project(Chronos CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

# Compile options
if(MSVC)
    add_compile_options(/W4)
else()
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# Main library
add_library(chronos_lib
    src/Date.cpp
    src/Task.cpp
    src/Project.cpp
    src/UrgencyCalculator.cpp
    src/TaskFilter.cpp
    src/UndoManager.cpp
    src/FileHandler.cpp
    src/ConsoleColor.cpp
    src/Menu.cpp
)
target_include_directories(chronos_lib PUBLIC include)

# Main executable
add_executable(chronos src/main.cpp)
target_link_libraries(chronos PRIVATE chronos_lib)

# Tests
enable_testing()
add_subdirectory(tests)
```

### 9.2. tests/CMakeLists.txt

```
include(FetchContent)
FetchContent_Declare(
```

```
    googletest
    GIT_REPOSITORY https://github.com/google/googletest.git
    GIT_TAG release-1.12.1
)
FetchContent_MakeAvailable(googletest)

# Unit tests
file(GLOB UNIT_TEST_SOURCES unit/*.cpp)
add_executable(unit_tests ${UNIT_TEST_SOURCES})
target_link_libraries(unit_tests PRIVATE chronos_lib gtest_main)
add_test(NAME unit_tests COMMAND unit_tests)

# Integration tests
file(GLOB INTEGRATION_TEST_SOURCES integration/*.cpp)
add_executable(integration_tests ${INTEGRATION_TEST_SOURCES})
target_link_libraries(integration_tests PRIVATE chronos_lib gtest_main)
add_test(NAME integration_tests COMMAND integration_tests)
```

---

## 10. Hướng Dẫn Build & Run

```
# Build
mkdir build && cd build
cmake ..
cmake --build .

# Run main
./chronos

# Run tests
ctest --output-on-failure

# Hoặc chạy trực tiếp
./tests/unit_tests
./tests/integration_tests
```

---

## 11. Tóm Tắt Yêu Cầu Đã Cover

| Yêu cầu đề bài     | Đã cover ở mục |
|--------------------|----------------|
| Lớp Task & Project | §3.3, §3.4     |
|                    |                |

|                              |                                        |
|------------------------------|----------------------------------------|
| Operator < (sort)            | §3.3 (operator<), §4.1 (sortByUrgency) |
| Operator ++ (gia hạn)        | §3.3 (prefix & postfix)                |
| std::stack cho Undo          | §5.1                                   |
| Time Paradox                 | §3.3 (constructor), §4.3, §7.2, §8.4   |
| Buffer Overflow (10000 task) | §3.4 (MAX_TASKS), §4.3, §7.3, §8.4     |
| Điểm Khẩn cấp                | §4.1                                   |
| Unit test                    | §7 (8 file)                            |
| Integration test             | §8 (4 file)                            |
| File save/load               | §5.2                                   |
| CLI menu                     | §6                                     |



### Cách dùng file này để vibecode:

1. Đưa toàn bộ file này cho Claude Code / Cursor.
2. Yêu cầu: "Implement toàn bộ codebase theo SPEC.md, tuân thủ chính xác signature, format, và pass mọi test trong §7-§8."
3. Build → chạy `ctest` → mọi test phải PASS.